

MotionJEPA: Preventing Temporal Feature Collapse by Capturing Visual Changes in Latent Space

Markus Karmann^{1,*} Shile Li^{2,*} Christian Internò^{3,1} Bruno Andreis^{1,4}
 David Klindt⁵ Randall Balestrieri^{6,7} Jindong Gu¹ Philip Torr^{1,4}
 Qi Zhang⁸ Peng-Tao Jiang⁸ Hao Zhang⁸ Bo Li⁸ Onay Urfalioglu²

¹University of Oxford ²vivo Tech Research GmbH ³Bielefeld University ⁴Slater Labs
⁵Cold Spring Harbor Laboratory ⁶Brown University ⁷AMI Labs
⁸vivo BlueImage Lab, vivo Mobile Communication Co., Ltd., China

*Equal contribution

Abstract

*Joint Embedding Predictive Architectures (JEPAs) are a promising paradigm for learning task-agnostic latent world models without visual reconstruction. However, standard JEPA training exhibits a strong inductive bias towards slow features, causing feature suppression and the collapse of latent representation. While inverse dynamics provides temporal anti-collapse, it relies on action labels and offers little incentive to embed general, unlabeled dynamics. We introduce **Difference Image and Single image embedding Regularization (DISReg)**, a novel regularizer that builds on an inverse-dynamics-style module that predicts temporal difference image embeddings without any pixel reconstruction loss, encouraging balanced static and dynamic feature learning. DISReg consists of a static term that shapes the distribution of the image embedding and encourages slow features, and a dynamic term, which, unlike direct regularization on the embedding, imposes no constraint on the image embedding's shape or distribution and instead only incentivizes that dynamic features be present. By integrating this regularizer into a standard JEPA, we establish our new architecture, **MotionJEPA**. Latent probing demonstrates that MotionJEPA produces more complete representations than other methods, and our trajectory analysis shows it maintains geometrically simple latent embeddings with low curvature. We further show that MotionJEPA improves downstream planning success under static-background distractors across four environments. Code available: <https://github.com/mkarmann/motion-jepa>.*

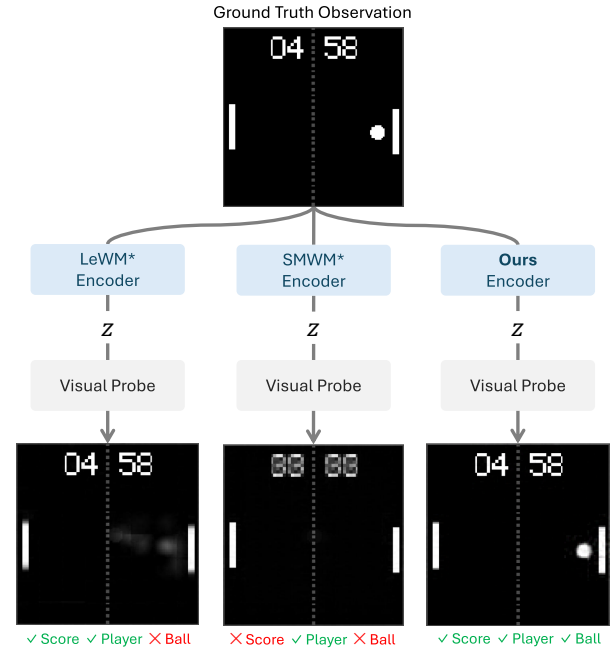


Figure 1. **Feature Suppression.** We compare three JEPA variants that differ in collapse-prevention mechanisms: LeWM [22] (SIGReg), SMWM [18] (Inverse Dynamics Module, IDM), and our MotionJEPA, which uses DISReg. The Pong environment contains three features: a fast-moving ball, two bats, and a slow-changing scoreboard. When trained from scratch, visual probes of the image embeddings show that SIGReg encodes the slowly changing scoreboard and bats but suppresses the ball. IDM encodes only the action-controlled bats. DISReg preserves all three.

1. Introduction

Joint Embedding Predictive Architectures (JEPAs) [20] have emerged as a powerful paradigm for representation learning by predicting future states directly within an abstract embedding space. However, a fundamental question in training JEPA architectures lies in determining exactly which features the model should capture. Previous works such as [24] suggest encoding features that are either controlled by the agent or can affect the agent, ignoring uncontrollable background features. Yet the optimal feature hierarchy is inherently problem- and task-dependent. Short-time planning operates on a short timescale, predicting fast-changing dynamics, whereas long-time planning operates on larger timescales with slower-changing features [20]. Although the primary objective of JEPA frameworks is to learn task-agnostic world models, standard formulations exhibit a well-documented inductive bias toward encoding slow features [26, 27]. In JEPA architectures, this trade-off is mainly controlled by the collapse prevention mechanism [26]. We illustrate this in Figure 1, comparing SIGReg-based LeWorldModel [22], inverse dynamics-based SMWM [18], and our DISReg-based MotionJEPA. The **Difference Image and Single image embedding Regularization (DISReg)** is a novel regularizer that encourages a complementary and balanced embedding of both static and dynamic features. DISReg is made of two terms. The static term shapes the distribution of the single-image embedding, encouraging the slow features that standard JEPA training already favors. The dynamic term instead predicts the embedding of the temporal difference image from consecutive image embeddings. This dynamic term does not constrain the shape or distribution of the image embedding and only incentivizes that features that change between frames are present, which we show results in geometrically simple latent trajectories with low curvature in Table 2.

Our main contributions are as follows:

- **DISReg**: A novel regularization term that incentivizes a balanced embedding of static and dynamic features by utilizing embeddings of the temporal difference images.
- **MotionJEPA**: A novel JEPA framework that utilizes DISReg, demonstrating better downstream performance in the presence of static distractors and robustness to feature collapse.
- A detailed collapse and latent trajectory analysis on a novel collection of three synthetic game environments (Pong, Dino, and Golf) with clearly separable, well-defined feature groups.
- Downstream control evaluation on experiments demonstrating the strengths of MotionJEPA in the presence of static distractors.

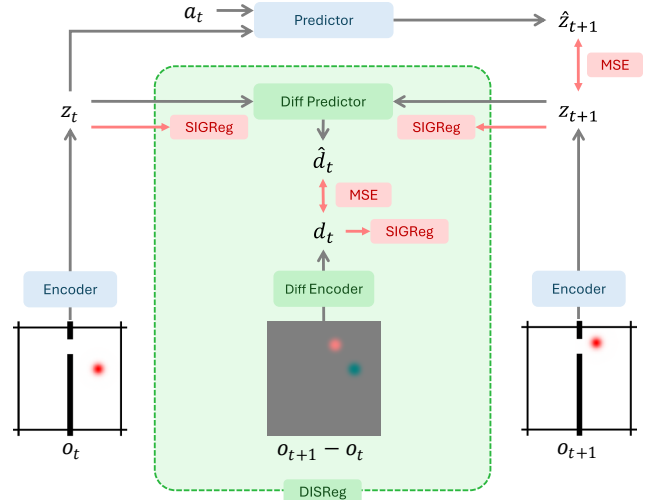


Figure 2. **MotionJEPA Overview.** Our novel regularizer DISReg utilizes a temporal difference image embedding d_t as visually grounded collapse prevention for temporally changing features. The core JEPA encoder and predictor architecture concept is illustrated in blue, and our DISReg in green. The JEPA forward prediction loss, as well as DISReg losses are highlighted in red.

2. Related Work

JEPA-style Latent World Models. The Joint Embedding Predictive Architecture (JEPA) is an architectural design that aims to learn latent representations without reconstructing the observation. As latent world models, these models learn to predict the dynamics of a system in latent space. The V-JEPA models [2, 5, 23] utilize a two-stage training paradigm by first pre-training on unlabeled video and then fine-tuning for the desired task. Recent work has further explored latent predictive models for control and planning, including LeWorldModel (LeWM) [22], which learns action-conditioned dynamics directly in representation space, and DINO-WM [32], which builds a world model on top of pretrained visual features for downstream planning tasks with Model Predictive Control (MPC). Temporal Difference in Vision (TDV) [9] does not require action labels and instead uses the pixel difference between consecutive frames to condition latent prediction and predict the corresponding change in representation space. Our work similarly exploits visually derived temporal change, but uses difference images as an auxiliary representation-learning objective to encourage the latent space to preserve changing scene information without requiring action labels.

Collapse Prevention in JEPA. Without a reconstruction loss, the main challenge of JEPAs is to avoid convergence to a trivial solution where all inputs map to a constant latent

representation. Contrastive methods use negative samples to explicitly push embeddings apart [8]. Non-contrastive methods utilize stop-gradient teachers [1, 2, 5, 9, 23]. Other approaches avoid negative pairs entirely, such as VICReg, which prevents informational collapse by applying explicit variance, invariance, and covariance regularization directly to the embeddings [4]. More recent methods avoid computing heavy covariance matrices by shaping the latent space directly into an isotropic Gaussian distribution. Specifically, LeJEPa [3] introduces SIGReg as its sole collapse-prevention loss, using the Epps-Pulley test [10] rather than the Sliced Wasserstein Distance (SWD) [6] to sketch the distribution. However, avoiding complete representational collapse does not ensure that the learned representation preserves all task-relevant factors of variation.

Feature Suppression and Slow-Feature Bias. Beyond complete collapse, self-supervised models may exhibit *feature suppression*, retaining only a subset of relevant information despite globally diverse representations [21, 25]. In temporal predictive learning, this can appear as a preference for slowly varying features. While JEPa performs on par with or better than reconstruction-based methods in settings where noise changes every time step, JEPa-style models fail when fixed noise or features are present. Sobal et al. [26] show that JEPas can encode fixed trajectory-specific distractors while neglecting faster foreground dynamics, a failure later reproduced across different noise timescales and predictive architectures [13]. Similar sensitivity to background changes has been observed in JEPa-based planning [28], while recent theory further connects LeJEPa’s temporal alignment objective to Slow Feature Analysis [19]. LeNEPA [7] applies SIGReg along the time dimension instead of the batch dimension to encourage temporal diversity in the learned representation.

Inverse dynamics provides another way to counteract this bias by forcing the representation to preserve information about action-relevant changes. Sensorimotor World Models (SMWM) [18] introduces an inverse model which predicts the action of two consecutive frames, given only the image embeddings. Experiments show that this additional module prevents collapse and enables the model to learn actual controllable degrees of freedom. Similarly, Delta-JEPa [31] also relies purely on the action prediction for collapse prevention, but provides the difference of two consecutive embeddings as input to the action decoder. A core limitation of inverse action prediction is that action labels are required. Additionally, these methods may be less suitable in scenarios where a given action causes a reaction in the observation with a time delay.

These observations motivate our evaluation setting, where we introduce static background distractors into

control environments to explicitly test whether the learned representation preserves changing, task-relevant scene information in the presence of strong slow features. Motivated by the same limitation, our method introduces a visually grounded temporal-difference objective that explicitly encourages the latent representation to preserve changing scene information without requiring action supervision.

3. Method

Problem formulation. We are given an offline dataset of trajectories $\mathcal{D} = \{(o_1, a_1, \dots, o_T)\}$ with visual observations $o_t \in \mathbb{R}^{3 \times H_{\text{obs}} \times W_{\text{obs}}}$ and continuous action labels $a_t \in \mathbb{R}^{D_a}$. The trajectories are not required to follow a task-specific policy and can be collected from any unknown or arbitrary behavior policy. The goal is to train a JEPa-style latent world model consisting of an encoder $\text{Enc}_\theta : \mathbb{R}^{3 \times H_{\text{obs}} \times W_{\text{obs}}} \rightarrow \mathbb{R}^{D_z}$ that maps individual observations o_t to a latent state $z_t \in \mathbb{R}^{D_z}$, and a forward predictor Pred_ϕ that estimates the deterministic state transition $\hat{z}_{t+1} = \text{Pred}_\phi(z_{t-H+1:t}, a_{t-H+1:t})$ where H is the size of the history context window. In the JEPa setting, this training process does not involve any reconstruction loss on the visual observations o_t , resulting in the primary challenge of preventing z_t from collapsing to trivial solutions [20].

Collapse prevention with DISReg. As illustrated in Fig. 2, our proposed DISReg introduces two new components, an encoder for encoding the temporal difference image $\text{DiffEnc}_\alpha : \mathbb{R}^{3 \times H_{\text{obs}} \times W_{\text{obs}}} \rightarrow \mathbb{R}^{D_d}$ and an inverse dynamics module-inspired predictor $\text{DiffPred}_\beta : \mathbb{R}^{D_z} \times \mathbb{R}^{D_z} \rightarrow \mathbb{R}^{D_d}$.

$$d_t = \text{DiffEnc}_\alpha(o_{t+1} - o_t) \quad (1)$$

$$\hat{d}_t = \text{DiffPred}_\beta(z_t, z_{t+1}) \quad (2)$$

DiffPred_β learns to predict the difference latent d_t of the difference image given the corresponding state embeddings z_t and z_{t+1} . This design has the advantage that we do not require action or state labels as targets, therefore introducing no bias towards labeled features.

Training losses. In contrast to a conventional JEPa design, our architecture has two latent representations z and d that are prone to collapse. We choose SIGReg [3] for collapse prevention, but other distribution regularization losses [6, 30] can also be utilized.

$$\mathcal{L}_z = \text{SIGReg}(z) \quad \mathcal{L}_d = \text{SIGReg}(d) \quad (3)$$

We establish the prediction loss between the temporal difference image embedding d_t and the respective

Algorithm 1. Pseudocode of DISReg. Inputs are an observations tensor $o \in \mathbb{R}^{N \times T \times 3 \times H_{\text{obs}} \times W_{\text{obs}}}$ with the per-image embeddings tensor $z \in \mathbb{R}^{N \times T \times D_z}$.

```
def DISReg(o, z):
    # 1. Compute temp. difference images
    o_current = o[:, :, :-1]
    o_next = o[:, :, 1:]
    o_diff = o_next - o_current

    # 2. Embed difference images
    d = DiffEnc(o_diff)

    # 3. Predict dynamics from latents
    z_current = z[:, :, :-1]
    z_next = z[:, :, 1:]
    d_pred = DiffPred(z_current, z_next)

    # 4. Estimate losses
    L_z = SIGReg(z)
    L_d = SIGReg(d)
    L_pred = (d - d_pred).pow(2).mean()

    # 5. Aggregate losses
    return 0.25*L_z + 2*L_d + 0.5*L_pred
```

predictor’s embedding $\hat{d}_t$ and aggregate them in $\mathcal{L}_{\text{DISReg}}$.

$$\mathcal{L}_{\text{pred}} = \text{MSE}(d_t, \hat{d}_t) \quad (4)$$

$$\mathcal{L}_{\text{DISReg}} = \lambda_z \mathcal{L}_z + \lambda_d \mathcal{L}_d + \lambda_{\text{pred}} \mathcal{L}_{\text{pred}} \quad (5)$$

Crucially, the scalar hyperparameters λ_z and λ_d control the category of information encoded in z . Setting $\lambda_d = 0$ allows d to collapse and DiffPred_β to trivially predict a constant, imposing no additional dynamic feature embeddings on z_t and z_{t+1} , which resembles LeWM [22]. By setting $\lambda_z = 0$, we maximize the information on embedded dynamics, creating a model that focuses on motions extracted from the temporal difference image. Because $\text{DiffPred}_\beta(z_t, z_{t+1})$ predicts the embedding of visual changes $\hat{d}_t$, minimizing $\mathcal{L}_{\text{pred}}$ results in both z_t and z_{t+1} learning transition-relevant information, preventing the collapse of the image embedding z without imposing additional constraints on the shape or distribution of the embedding. Finally, we join the forward prediction objective of JEPA with our DISReg to obtain the total loss function of MotionJEPA:

$$\mathcal{L}_{\text{MotionJEPA}} = \mathcal{L}_{\text{DISReg}} + \text{MSE}(z_{t+1}, \hat{z}_{t+1}) \quad (6)$$

We provide a pseudocode implementation of the DISReg computation in Algorithm 1, using the default weights applied throughout our experiments.

4. Experiments

The experiments are organized into two main sections. Sec. 4.1 investigates collapse scenarios on game environments with slow and static features by probing the latent embedding z and further analyzing its path straightness. Sec. 4.2 demonstrates our method’s performance on downstream tasks with static distractors. Throughout all experiments, we use the same set of hyperparameters for MotionJEPA’s DISReg: $\lambda_z = 0.25$, $\lambda_{\text{pred}} = 0.5$, and $\lambda_d = 2$.

Baselines. We compare our method with previous end-to-end JEPA-style latent world models, namely **LeWM** [22] and **SMWM** [18] with default SIGReg weight of 0.09 for LeWM and inverse prediction weight of 1.0 for SMWM. We additionally provide altered versions of LeWM with common practice modifications used in the literature to improve robustness against feature collapse. **LeWM-Time** applies SIGReg along the time dimension [7], **LeWM-Flat** applies SIGReg along the flattened batch and time dimension, and **LeWM-Detached** adds a StopGradient operator in the JEPA predictor loss target embedding $\text{MSE}(\text{StopGradient}(z_{t+1}), \hat{z}_{t+1})$ [1, 5, 12]. Besides these baselines, we also include **ForwardOnly**, which has only the forward prediction loss and no anti-collapse mechanism, and **Random**, with fixed, randomly initialized weights, showing what information is present in the latent at initialization that might be suppressed during training. Finally, we also perform a hyperparameter search for each baseline environment combination to demonstrate that the collapse is not an issue of hyperparameter selection but a fundamental limitation of JEPA anti-collapse mechanisms [26]. We mark such models with a star * throughout this work.

4.1. Collapse and Latent Embedding Analysis

Environments. We designed three fully synthetic, deterministic game environments with clearly separable feature groups. Samples for each environment are generated on the fly during training and evaluation to prevent overfitting¹. Initial states and actions are sampled randomly for each episode to minimize correlations between features. This reduces the risk of shortcut learning [11]; for example, in Pong, a probe estimating the bat’s position solely from the ball’s location would falsely suggest that bat information is encoded when it is not.

We propose three environments of increasing dynamical complexity: **Pong**, where the agent controls both bats to hit the ball; **Dino**, a 2D side-scroller where an agent must jump over a cactus; and **Golf**, in which the player navigates a ball

¹Sampling is seeded to ensure that all methods observe identical training and evaluation sequences for a given seed.

(a) Offline Residual MLP Probe NMSE $\downarrow$ of Image Embedding z

Method	Pong				Dino						Golf				
	avg.	ball	bats	score	avg.	cactus	clouds	dino	hearts	sky	avg.	bars	cross	ball	wood
ForwardOnly	1.058	1.260	1.001	0.930	0.949	1.003	1.002	1.002	1.031	0.511	1.980	1.233	1.558	1.077	4.324
Random	0.116	0.143	0.093	0.115	0.361	0.833	0.535	0.116	0.031	0.004	0.776	0.857	0.296	0.989	0.653
SMWM [18]	0.657	1.001	0.002	1.002	0.649	0.434	1.075	0.028	0.787	0.071	0.727	1.000	0.986	0.002	1.003
LeWM [22]	0.658	0.999	1.002	0.003	0.394	0.769	0.248	0.983	0.012	0.005	0.763	1.002	1.001	0.942	0.084
LeWM-Time	1.077	1.100	1.071	1.063	0.913	1.003	0.992	1.002	1.029	0.248	1.053	1.008	1.014	1.004	1.195
LeWM-Detached	0.171	0.174	0.331	0.007	0.645	1.002	0.727	1.002	0.017	0.006	0.811	0.924	0.683	0.930	0.572
LeWM-Flat	0.284	0.890	0.014	0.005	0.331	0.522	0.593	0.002	0.006	0.007	0.694	1.002	0.379	0.993	0.052
SMWM*	0.651	0.983	0.002	1.003	0.606	0.398	1.005	0.003	0.775	0.069	0.727	1.000	0.986	0.002	1.003
LeWM*	0.061	0.163	0.018	0.012	0.282	0.990	0.059	0.751	0.011	0.012	0.760	1.001	1.002	0.909	0.110
LeWM-Time*	0.687	0.022	0.972	1.002	0.697	0.005	1.004	0.003	0.971	1.004	0.985	0.972	0.992	0.984	1.002
LeWM-Detached*	0.048	0.103	0.038	0.008	0.332	1.005	0.073	0.959	0.011	0.017	0.745	1.002	1.002	0.908	0.048
LeWM-Flat*	0.008	0.012	0.004	0.008	0.301	0.650	0.498	0.002	0.007	0.016	0.299	0.317	0.059	0.312	0.377
MotionJEPA (Ours)	0.004	0.005	0.003	0.004	0.021	0.011	0.037	0.006	0.006	0.004	0.061	0.095	0.083	0.033	0.033

(b) Offline Residual MLP Probe NMSE $\downarrow$ of Predicted Rollout $\hat{z}$

Method	Pong				Dino						Golf				
	avg.	ball	bats	score	avg.	cactus	clouds	dino	hearts	sky	avg.	bars	cross	ball	wood
ForwardOnly	1.010	1.041	1.006	0.992	1.081	1.019	1.006	1.004	1.017	1.634	1.003	1.002	1.001	1.005	1.005
Random	0.554	0.721	0.478	0.507	0.617	0.974	0.764	0.631	0.279	0.026	0.934	0.995	0.913	0.998	0.781
SMWM [18]	0.630	1.001	0.002	1.002	0.686	0.839	1.002	0.092	0.933	0.076	0.729	1.001	1.001	0.006	1.002
LeWM [22]	0.644	1.002	1.002	0.002	0.407	0.856	0.251	0.995	0.017	0.005	0.767	1.002	1.001	0.947	0.094
LeWM-Time	1.001	1.003	0.998	1.002	1.106	1.011	1.009	1.003	1.026	1.845	1.003	1.006	1.006	1.000	1.002
LeWM-Detached	0.239	0.397	0.347	0.007	0.646	0.988	0.732	1.002	0.028	0.006	0.855	0.967	0.821	0.930	0.618
LeWM-Flat	0.278	0.968	0.039	0.005	0.396	0.877	0.633	0.055	0.009	0.007	0.722	1.002	0.590	0.994	0.061
SMWM*	0.630	1.001	0.001	1.002	0.671	0.801	0.999	0.075	0.878	0.079	0.729	1.001	1.001	0.006	1.002
LeWM*	0.138	0.471	0.020	0.009	0.334	0.991	0.060	1.003	0.018	0.012	0.763	1.002	1.002	0.913	0.117
LeWM-Time*	0.868	0.507	1.002	1.002	0.710	0.120	1.004	0.020	0.959	1.003	0.997	0.997	0.995	0.991	1.002
LeWM-Detached*	0.129	0.395	0.053	0.008	0.343	0.988	0.078	1.002	0.020	0.014	0.746	1.002	1.001	0.909	0.051
LeWM-Flat*	0.054	0.183	0.004	0.007	0.353	0.923	0.524	0.066	0.013	0.018	0.448	0.540	0.320	0.402	0.425
MotionJEPA (Ours)	0.036	0.122	0.004	0.005	0.055	0.243	0.040	0.051	0.007	0.005	0.164	0.262	0.348	0.065	0.037

Table 1. **Probing Results on Game Environments.** We use high-capacity MLP probes with residual connections to probe the latent embedding z for predicting environment features and report the NMSE averaged over three runs. (a) Probes the image embedding directly from the encoder; (b) probes on rolled-out states of a rollout length of 5. Runs marked with * are from a hyperparameter search. For (a), based on the results of the random weights model on Pong, NMSE values ≤ 0.1 are marked in green as not collapsed; otherwise, red for collapsed. Best results in each column are highlighted in bold and second best underlined.

toward a hole while avoiding moving obstacles. We design the true state dimension of each environment (excluding velocities: Pong: 6, Dino: 8, Golf: 6) to be significantly lower than the latent embedding dimension D_z of 192 to ensure that the latent dimensionality is not a bottleneck that artificially induces collapse. Further implementation details for each environment are provided in Sec. 7.

Evaluation Setup. We follow [26] and analyze the collapse of features by probing the learned image embedding z . To keep the comparison fair, all methods use the same image encoder, latent forward predictor, training schedule, and optimizer from [22]. After training from scratch on 10M randomly sampled episodes, we freeze

the base model, train probes on 1M training samples, and evaluate those probes on 10k episodes.

For the collapse analysis, we choose higher-capacity MLP probes with residual connections instead of linear probe, as we are not interested in the linear separability or ease of accessibility of features, but rather in measuring the general presence or suppression of information in the latent embedding z . For example, a world model which uses a non-linear, highly correlated representation in z might still be functional, yet linear probes might fail to decode that information.

We report Normalized Mean Squared Error (NMSE) for each probe. An NMSE of 1.0 corresponds to predicting the evaluation target mean, indicating a complete failure to

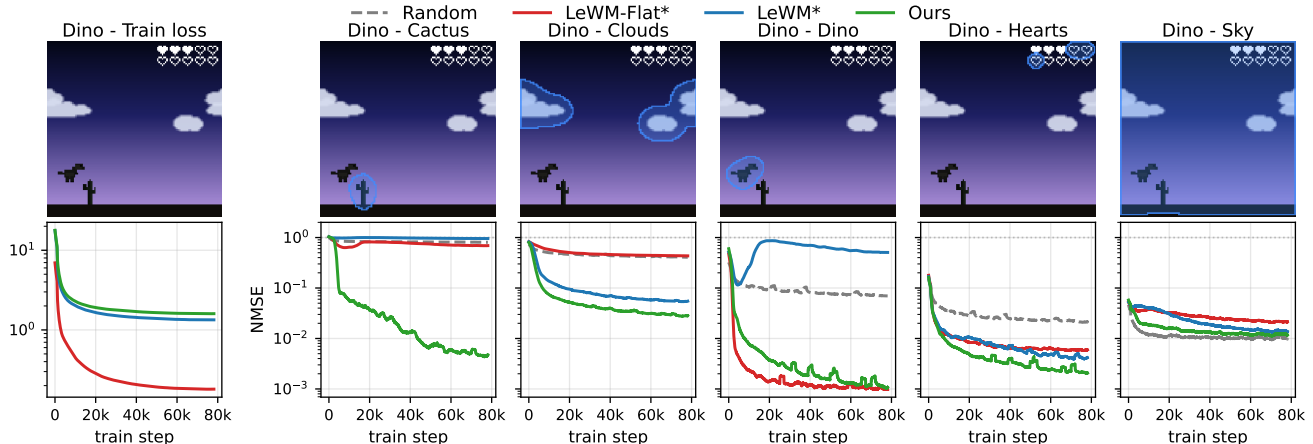


Figure 3. **State Probing during Training.** We train a detached state decoder alongside training the JEPA model. The first column of each environment is the training loss (log scale). The remaining curves are per-feature probe NMSE. Image areas affected by the corresponding feature are highlighted in blue. Models marked with * are optimized with a hyperparameter search.

extract useful information from z . We train and evaluate each method-environment pair for three seeds and report the average results. We provide more details about the training and evaluation process in Sec. 7.

Probing Results. Tab. 1 shows the MLP probing results on Pong, Dino, and Golf for both the encoder embeddings z as well as predicted rollouts $\hat{z}$. The goal is to obtain the lowest average loss across all categories for each environment, indicating stable and balanced learning of all features². Looking at the results of probing the image embedding z , as expected, the control method ForwardOnly collapsed on all three tasks, resulting in an NMSE greater than or about 1.0 for all features except the sky color of Dino, with 0.511, which is arguably the easiest feature of these environments. Switching to the Random model, we observe that even though the weights are random, the probes are still able to extract a significant amount of information, with an NMSE average of 0.116 for Pong and even 0.004 for the sky in the Dino environment. This shows that even though the feature information is accessible at the start of training, models actively suppress it during training.

Using default training parameters for LeWM results in clear collapses across all three environments, focusing only on slow or static features such as the score in Pong. While the hyperparameter-optimized version LeWM* improves the average probing loss on a simple task such as Pong from 0.658 to 0.061, in the presence of more dynamic features, as in Dino and Golf, such hyperparameter optimization does not significantly improve results (see grid search in the

appendix in Fig. 7), highlighting a core limitation of not regularizing along the time dimension.

Both the LeWM-Flat* and LeWM-Time* variants add distribution constraints on the time dimension. LeWM-Time is the only model variant besides ours that successfully encodes the cactus in Dino, yet completely fails in the presence of more complex moving features in Golf. SMWM uses an inverse dynamics module that predicts the action of two consecutive states. It performs well on states which are directly impacted by actions but collapses on other task-relevant information.

Our method achieves the lowest average feature probing loss across all three environments for both image embedding and rollout prediction while using the **same set of hyperparameters** across all three environments. MotionJEPA is also the only method that achieves a balanced probing loss below 0.1 on all categories, indicating robustness against feature collapse. Especially for the bar obstacle category of Golf, where MotionJEPA obtains 0.095 while all other methods suppress this category with losses higher than 0.3. We also provide results with linear probes in Tab. 5 of the appendix, in which our model also achieves the lowest average probing loss across all environments.

When does collapse happen? In Fig. 3 we show the training loss curves with additional online MLP probes for the Dino environment. The probes are trained on the z embedding with a stop-gradient operator to prevent information leaking to the encoder. The static sky and slow-changing hearts are the easiest features, quickly saturating at about 10k steps to 0.01 NMSE. We also observe that the random weights model is encoding these

²The average column presents a weighted mean, where each feature group’s weight corresponds to its dimensionality.

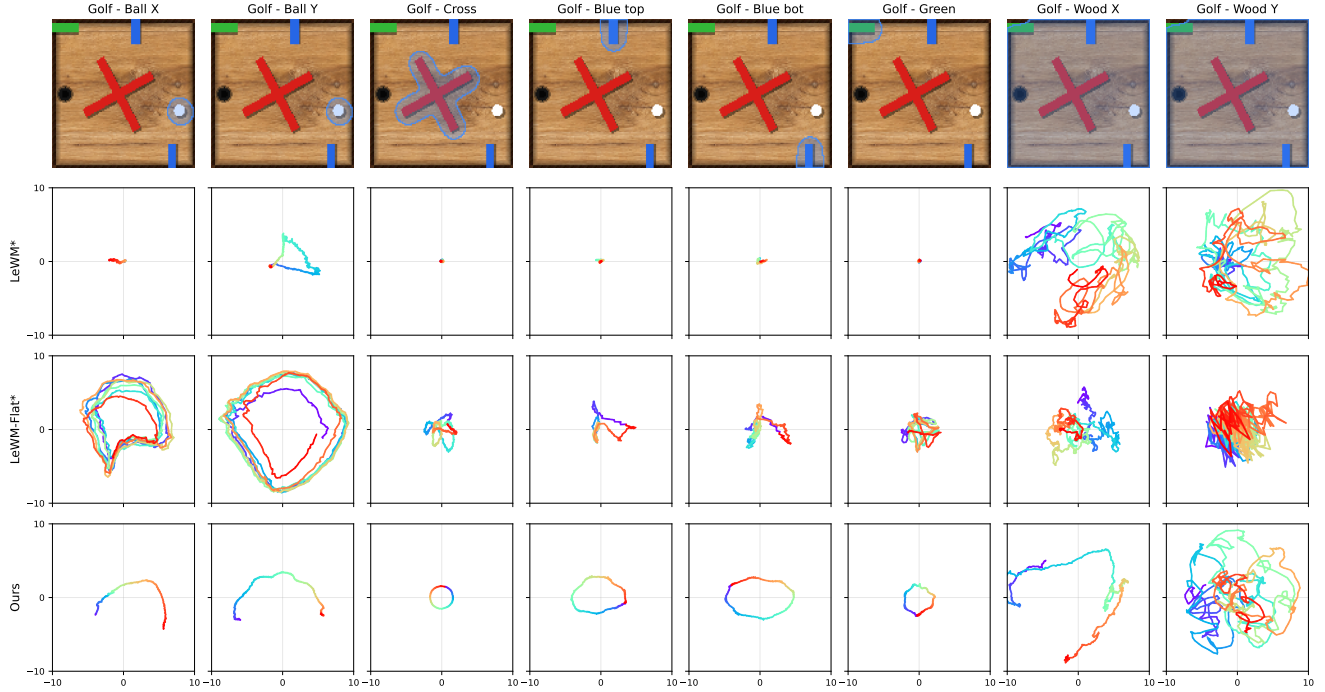


Figure 4. **Latent Trajectories.** We visualize how individual state features map into the latent space. For each subplot, we vary a single feature across a 1D range while holding the remaining state variables fixed, render the resulting sequence into images, and encode them into latent representations z . We then project these latent trajectories onto their first two principal components without variance rescaling. We further color the state range in rainbow colors from 0: violet to 1: red.

features as well. For LeWM, in which SIGReg is regularizing the distribution along the batch dimension, the slow features of the sky, hearts, and clouds suppress the embedding of the cactus and dinosaur. This is especially visible for the dinosaur, as the probing loss first decreases to approximately 0.1 at 5k iterations and then increases back to 1.0 at 20k iterations, from which it is not able to recover.

LeWM-Flat, which enforces a distribution across both batch and time dimensions, seeks variance along the time dimension of the embedding as well and therefore quickly picks up the dinosaur. This is probably because its limited movement of jumping is very simple, but provides consistent temporal change.

Crucially, the collapse is not visible in the training loss (see first column), demonstrating that monitoring loss alone is insufficient to diagnose collapse without explicit latent probing or downstream evaluation.

Latent Path Straightness. The temporal straightening hypothesis [14] states that useful representations turn curved input dynamics into near-straight latent paths. It has proven informative in practice, as a signal for identifying AI-generated video [15], for judging physical plausibility [16], and as a planning objective [29]. We show empirical results of the latent path straightness in

Tab. 2. SMWM achieves the straightest latent trajectories close to the ideal straight-line value of 1.0 in both Pong and Golf, as it has no distribution regularization on z . Introducing distribution regularization along the time axis, such as in variants LeWM-Time* and LeWM-Flat* causes the opposite effect, significantly increasing the latent embedding curvature to above 20 for both Pong and Golf. As our method is architecturally closer to SMWM’s IDM module, we observe straighter paths than all distribution regularization methods on Pong and Golf.

This can also be observed in Fig. 4, where we show example latent trajectories of each feature of the golf environment. Our method produces nearly straight latent embeddings for ball position and single-loop embeddings for the circular cross and bar obstacles³. LeWM collapses all obstacles into almost a single point while showing high-curvature embeddings in the distracting wood background. While the representations of the LeWM-Flat variant do not collapse into a single point, the embedding has more curvature and entanglement, making it difficult to separate and predict individual features. We provide additional latent trajectory visualization of other environments and methods in the appendix.

³The bar obstacles reappear on the opposite side if exiting the image and therefore result in a looped trajectory.

Method	Pong	Dino	Golf
ForwardOnly	45.514	4.045	26.412
Random	25.278	13.660	26.842
SMWM [18]	1.207	7.557	1.078
LeWM [22]	5.170	4.826	8.471
LeWM-Time	29.911	4.955	27.190
LeWM-Detached	3.067	18.372	15.455
LeWM-Flat	3.031	12.986	5.298
SMWM*	1.427	8.631	1.078
LeWM*	8.170	7.755	8.191
LeWM-Time*	23.376	9.979	38.839
LeWM-Detached*	4.111	8.806	7.935
LeWM-Flat*	22.159	15.610	26.663
MotionJEPa (Ours)	2.756	5.789	3.305

Table 2. **Latent path straightness.** We calculate the mean encoder arc length over the chord length for 100 random linear state-space interpolations. Lower is straighter, with 1.0 being a straight line.

4.2. Downstream Tasks

As in LeWorldModel [22], probing is intended primarily as an analysis of representation content rather than to substitute for closed-loop control evaluation. We therefore take the experimental setup of LeWM [22] and evaluate the model’s success rate in the presence of slow features.

Environments. All four environments originally contain no static or slow visual features. To introduce a static-background distractor, we replace the original background with a wood-texture image, mimicking games played on a wooden table. For each rollout, a different crop of the texture image is sampled and kept fixed throughout the rollout. For PushT and TwoRoom, the original white background pixels are directly replaced with the wood texture. For Reacher and OG-Cube, we first estimate a static-background mask using the pixel-wise median over the training dataset and replace the corresponding regions with the texture. Example observations from all four environments are shown in Fig. 5. All models are trained for 10 epochs with a batch size of 64 using AdamW, with a learning rate of 5×10^{-5} and weight decay of 10^{-3} .

Evaluation and Results. For evaluation, we use CEM-based planning following the protocol of the original LeWorldModel paper [22]. For IDM, we use our own implementation of inverse-dynamics supervision, predicting the raw action from $\Delta z_t = z_{t+1} - z_t$ with a three-layer MLP. This follows the Δz input formulation of DeltaJEPa [31], while SMWM [18] similarly employs inverse-dynamics supervision with a different latent input

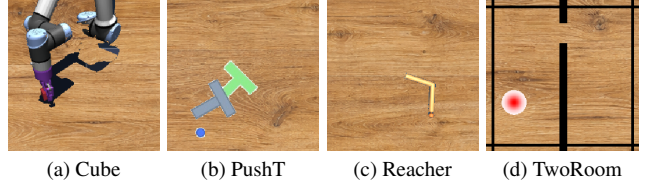


Figure 5. Example observations from the four LeWorldModel environments with a wood-texture background used as a static visual distractor. A different crop of the wood texture is sampled for each rollout while remaining fixed within the rollout.

parameterization. As an additional baseline, we evaluate LeWM-Flat, which applies SigReg after flattening the batch and temporal dimensions. Initial states are randomly sampled from the test split, and we evaluate planning horizons of 25 and 50 steps. For the 25-step setting, we follow the CEM planning configuration of [22], using 300 candidate sequences, 30 elites, and the same environment-specific optimization budget. With a frame skip of 5, CEM plans five model steps ahead, corresponding to 25 environment steps. For the 50-step setting, we first plan and execute a 25-step trajectory, then replan from the resulting state for the remaining 25 steps using the same CEM configuration. The results are summarized in Table 3. Overall, the original LeWM baseline degrades substantially in the presence of the static-background distractor, particularly on PushT and Reacher, while LeWM-Flat provides a stronger baseline but still remains clearly below IDM and our method in mean performance. IDM improves robustness across the four environments, while our standalone method achieves stronger overall mean performance, with particularly large gains on Cube and Reacher under the longer 50-step planning horizon. Combining our objective with IDM yields the best overall results, further indicating that the two training signals are complementary. On TwoRoom, both approaches achieve near-saturated performance, whereas the original LeWM baseline remains considerably lower. We further probe task-relevant states from the frozen representations on the four LeWorldModel benchmarks with static wood-texture distractors. Our method substantially improves state recoverability over LeWorldModel, consistent with its stronger downstream planning performance. Detailed linear/MLP probing and reconstruction results are provided in Sec. 10.

Ablation Studies. For the four LeWM benchmark tasks, our method builds on LeWM and introduces an additional prediction objective using temporal difference images. As shown in Table 3, removing the regularization on the original latent representation ($\lambda_z = 0$) degrades performance, indicating that preserving global state

Method	Cube	PushT	Reach.	2Room	Mean
25-step planning					
<i>Baseline methods</i>					
LeWM [22]	43.6	4.0	5.2	28.8	20.4
LeWM-Flat	55.2	14.4	49.2	48.0	41.7
IDM [18, 31]	76.0	83.6	49.2	100.0	77.2
<i>Our method</i>					
Ours ($\lambda_z = 0$)	60.4	76.8	24.8	99.6	65.4
Ours	<u>78.0</u>	78.4	<u>71.2</u>	99.6	<u>81.8</u>
Ours + IDM	80.8	<u>82.4</u>	82.4	100.0	86.4
50-step planning					
<i>Baseline methods</i>					
LeWM [22]	24.4	2.4	2.0	18.4	11.8
LeWM-Flat	42.4	2.0	60.4	22.8	31.9
IDM [18, 31]	53.2	26.0	62.0	<u>96.0</u>	59.3
<i>Our method</i>					
Ours ($\lambda_z = 0$)	48.0	21.2	33.6	94.0	49.2
Ours	63.2	<u>28.0</u>	<u>98.0</u>	92.0	<u>70.3</u>
Ours + IDM	<u>61.2</u>	29.6	98.4	97.2	71.6

Table 3. Planning success rate (%) under static-background distractors. Our method gives the best standalone mean performance, while combining it with IDM yields the best overall results.

information remains important alongside change-aware supervision. We further apply the same difference-image objective to the IDM-based LeWM variant and observe consistent improvements in 25-step planning, showing that the proposed objective is complementary to inverse-dynamics supervision and can serve as a useful add-on to existing world-model training.

5. Limitations

DISReg introduces two additional modules, DiffEnc_α and DiffPred_β , both increasing the parameter count and compute cost relative to a plain JEPA baseline. However, the training loss curves in Fig. 3 show that collapse is not resolved simply by training longer or with more capacity. The baselines plateau in NMSE despite continued training, indicating that feature suppression is not an issue of compute budget. Furthermore, the modules introduced by DISReg are only relevant during training and are discarded afterward, adding no computational overhead during inference.

6. Conclusion

We introduced DISReg, a regularizer that encourages JEPA world models to encode dynamic features by predicting an embedding of the temporal difference image, without action labels or pixel reconstruction. Integrated in MotionJEPA, our model captures both static and dynamic

features more completely than prior methods, with straighter, less entangled latent trajectories, using one fixed hyperparameter set throughout. This translates to improved downstream planning under static-background distractors. Overall, encouraging dynamics through prediction rather than direct latent constraints offers a simple, effective route to collapse-resistant JEPA world models.

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023. 3, 4
- [2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba, Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025. 2, 3
- [3] Randall Balestriero and Yann LeCun. Lejpa: Provable and scalable self-supervised learning without the heuristics, 2025. 3
- [4] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022. 3
- [5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. 2, 3, 4
- [6] Nicolas Bonneel, Julien Rabin, Gabriel Peyré, and Hanspeter Pfister. Sliced and Radon Wasserstein barycenters of measures. *Journal of Mathematical Imaging and Vision*, 51(1):22–45, 2015. 3
- [7] Alexander Chemeris, Ming Jin, and Randall Balestriero. Lenepa: No-augmentation next-latent prediction for time-series representation learning, 2026. 3, 4
- [8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, pages 1597–1607. PMLR, 2020. 3
- [9] Ninad Daithankar, Alexi Gladstone, Yann LeCun, and Heng Ji. You don’t need strong assumptions: Visual representation learning via temporal differences, 2026. 2, 3
- [10] Thomas W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983. 3
- [11] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and

- Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. 4
- [12] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, Bilal Piot, koray kavukcuoglu, Remi Munos, and Michal Valko. Bootstrap your own latent - a new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, pages 21271–21284. Curran Associates, Inc., 2020. 4
- [13] Paarth Gulati and Ilya Nemenman. Contrast encodes inductive bias: separating slow noise from dynamics in predictive representation learning. *arXiv preprint arXiv:2606.07770*, 2026. 3
- [14] O J Hénaff, R L T Goris, and E P Simoncelli. Perceptual straightening of natural videos. *Nature Neuroscience*, 22(6): 984–991, 2019. 7
- [15] Christian Internò, Robert Geirhos, Markus Olhofer, Sunny Liu, Barbara Hammer, and David Klindt. AI-generated video detection via perceptual straightening. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. 7
- [16] Christian Internò, Alexander Pondaven, Habon Issa, Fabio Pizzati, Francesco Pinto, Markus Olhofer, Ivan Laptev, Philip Torr, Eero P. Simoncelli, Barbara Hammer, and David Klindt. Geophys: The geometry of physical plausibility, 2026. 7
- [17] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, Lille, France, 2015. PMLR. 1
- [18] Petr Ivashkov, Randall Balestriero, and Bernhard Schölkopf. Sensorimotor world models: Perception for action via inverse dynamics, 2026. 1, 2, 3, 4, 5, 8, 9, 7, 10, 11
- [19] David Klindt, Yann LeCun, and Randall Balestriero. When does lejepa learn a world model? *arXiv preprint arXiv:2605.26379*, 2026. 3
- [20] Yann LeCun. A path towards autonomous machine intelligence, 2022. OpenReview position paper. 2, 3
- [21] Tianhong Li, Lijie Fan, Yuan Yuan, Hao He, Yonglong Tian, Rogerio Feris, Piotr Indyk, and Dina Katabi. Addressing feature suppression in unsupervised visual representations. In *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1411–1420. IEEE, 2023. 3
- [22] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels, 2026. 1, 2, 4, 5, 8, 9, 3, 7, 10, 11
- [23] Lorenzo Mur-Labadia, Matthew Muckley, Amir Bar, Mido Assran, Koustuv Sinha, Mike Rabbat, Yann LeCun, Nicolas Ballas, and Adrien Bardes. V-jepa 2.1: Unlocking dense features in video self-supervised learning, 2026. 2, 3
- [24] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *ICML*, 2017. 2
- [25] Joshua Robinson, Li Sun, Ke Yu, Kayhan Batmanghelich, Stefanie Jegelka, and Suvrit Sra. Can contrastive learning avoid shortcut solutions? *Advances in neural information processing systems*, 34:4974–4986, 2021. 3
- [26] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features. In *NeurIPS Workshop on Self-Supervised Learning*, 2022. 2, 3, 4, 5
- [27] Enigma Team. The obsessed encoder, 2026. Accessed: 2026-08-29. 2
- [28] Leonardo F Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026. 3
- [29] Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. In *Forty-third International Conference on Machine Learning*, 2026. 7
- [30] Haiyu Wu, Randall Balestriero, and Morgan Levine. Visreg: Variance-invariance-sketching regularization for jepa training, 2026. 3
- [31] Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-jepa: Learning action-sensitive world models via latent difference decoding, 2026. 3, 8, 9
- [32] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024. 2

MotionJEPA: Preventing Temporal Feature Collapse by Capturing Visual Changes in Latent Space

Supplementary Material

7. Implementation Details

7.1. Model and Component Architectures

Encoder and Predictor Architecture. We use the same encoder and predictor architectures as [22]. The encoder is a ViT-Tiny Vision Transformer with a patch size of 14×14 pixels. As in [22], the $D_z = 192$ dimensional embedding vector is obtained from the [CLS] token of the final layer, followed by a 1-layer MLP with Batch Normalization [17]. The predictor is a six-layer causal transformer with 16 attention heads and learned positional embeddings [22].

DISReg Components. We use the same architecture for the DiffEnc_α as for Enc_θ . The difference predictor DiffPred_β is a three-layer MLP with hidden layers of size 512 that uses LayerNorm and GELU after all but the final linear layer. The input is a concatenated vector of the two consecutive embeddings z_t and z_{t+1} .

Residual MLP Probe. The residual MLP probe has the highest capacity and is intended to check the general presence of information, not ease of accessibility. It is an MLP of depth 6 and width 128 with residual connections between every second layer and pre-activation batch normalization.

MLP Probe. We train a two-layer MLP on top of the frozen latent representation to measure non-linear accessibility of the states, comprising a linear layer with 512 hidden units, a GELU activation, and a final linear output layer. Angular targets are represented by their sine and cosine values and trained with MSE; angular error is computed after decoding the predicted angle.

Linear Probe. We train a single affine linear layer, including a bias term, on top of the frozen latent representation. No hidden layers, normalization, or activation functions are used. For LeWM benchmark problems, angular targets are represented by their sine and cosine values and trained with MSE; angular error is computed after decoding the predicted angle.

7.2. Game Environments

For all games, the states and actions are normalized to the range $[0, 1]$. Initial states for the first frame and actions are sampled independently from a uniform distribution. These

states are then simulated deterministically with the given actions to generate a total of 4 states and corresponding frames. Sample episodes of each dataset are shown in Fig. 6.

7.2.1. Pong

State Variables: left bat position, right bat position, ball x-position, ball y-position, ball x-velocity, ball y-velocity, score left player, score right player.

Actions: left bat velocity, right bat velocity.

The positions of the ball and the players' bats are also uniformly initialized. If a player hits the ball, the ball's trajectory changes towards the opponent. The change in angle is dependent on the bat's position, and the horizontal velocity accelerates with each hit up to a maximum velocity. If one player scores a goal, the score of the respective player increases by one, and the ball is reset to the center, moving towards the player who was scored against. The task requires the model to estimate the ball's velocity in order to predict the ball's next position.

7.2.2. Dino

State Variables: dinosaur y-position, dinosaur y-velocity, cactus x-position, cactus x-velocity, normalized number of hearts, sky color, clouds height, cloud 0 scroll, cloud 1 scroll, cloud 2 scroll.

Actions: jump.

A side scroller in which the goal is to time the dinosaur's jumps to avoid upcoming cacti. Clouds each move with an individual fixed velocity for each episode. If the dinosaur collides with one of the cacti, the number of hearts is reduced by 1 (corresponding to a change of 0.1 in the normalized state vector). Once all lives are depleted, the game freezes for all remaining frames.

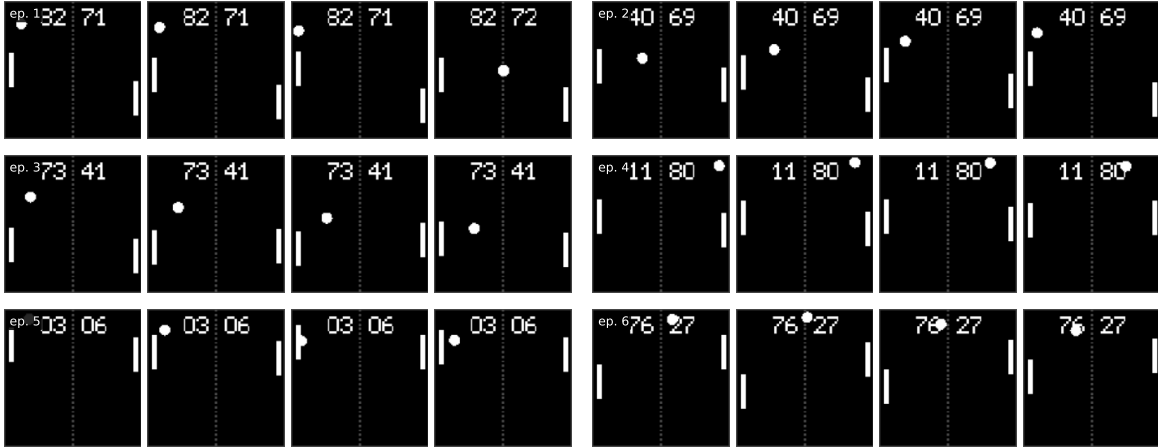
7.2.3. Golf

State Variables: ball x-position, ball y-position, cross angle, cross angular velocity, blue bar 1 x-scroll, blue bar 1 scroll x-velocity, blue bar 2 x-scroll, blue bar 2 x-scroll velocity, green bar y-scroll, green bar y-scroll velocity.

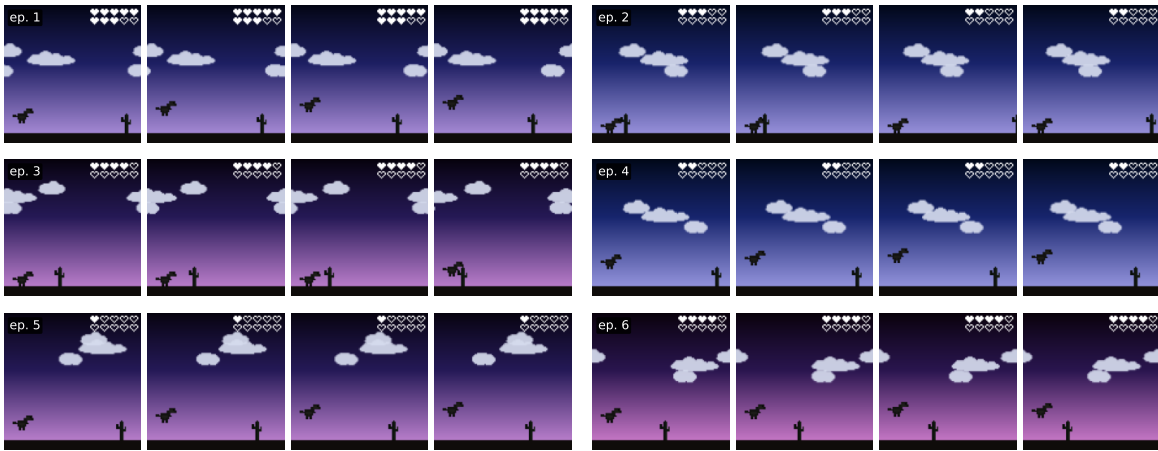
Actions: ball x-velocity, ball y-velocity.

In this game, the player navigates a ball through a 2D course of moving obstacles. Each obstacle is initialized at a random position, together with a random velocity that is fixed for each episode. The bars, when exiting the screen on one side, reappear on the other side of the screen. Colliding

(a) Pong



(b) Dino



(c) Golf

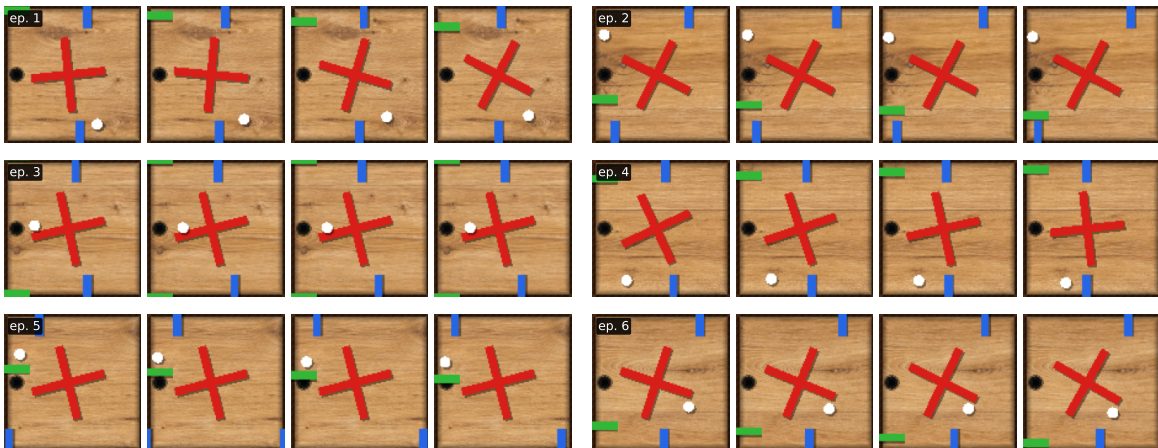


Figure 6. **Randomly sampled episodes of game environments.** We show six randomly sampled 4-frame episodes of each environment. Each episode is the same length as used during training. Episode one of Pong shows the right player scoring, and episode two of Dino demonstrates the player losing one heart by running into the cactus.

with obstacles stops the game, freezing it for the remaining frames and ignoring all action inputs. As the red cross is symmetric to one quarter of a full rotation, we normalize the cross angle to a range of 0 to 90 degrees.

7.3. Details for Training on Game Environments

Training and probe fitting use mixed-precision *bfloat16* automatic mixed precision and *torch.compile*.

Pre-training on Game Environments. Experiments on the four game environments are conducted at a resolution of 112×112 pixels with no skip frames (i.e., simple next-frame prediction). We train the models using a history size of three on 10 million four-frame episodes with a batch size of 128, resulting in approximately 78k training steps. We use the AdamW optimizer with a weight decay of 10^{-3} . For the learning rate, we employ a linear-warmup cosine-annealing schedule, matching the LeWM [22] defaults: the learning rate rises linearly from 0 to 5×10^{-5} over the first 1% of training (781 steps), then decays to zero following a cosine schedule over the remaining steps. This schedule is stepped after every optimizer update.

Fitting Probes on Game Environment Models. After pre-training, we load and freeze the weights of the resulting encoder Enc_θ and predictor Pred_ϕ . We generate 1 million eight-frame episodes and, for each episode, embed the first three frames into latent vectors $z_{1:3}$ using the encoder Enc_θ . Next, we use the first three embeddings $z_{1:3}$ as context and apply the predictor Pred_ϕ autoregressively with a sliding window of size three and conditioned on the episode’s ground-truth actions to generate a rollout of five predicted embeddings $\hat{z}_{4:8}$. Either image embeddings $z_{1:3}$ or predicted rollout embeddings $\hat{z}_{4:8}$ are then used as inputs for the Residual MLP or Linear Probes. This results in a total of four probes: a Linear Probe on image embeddings, a Linear Probe on rollout embeddings, a Residual MLP Probe on image embeddings, and a Residual MLP Probe on rollout embeddings. The training target for each probe is the state vector detailed in Sec. 7.2. We use the AdamW optimizer with a weight decay of 10^{-3} , a fixed learning rate of 3×10^{-3} , and a batch size of 128. We evaluate each probe’s performance on an additional 10k episodes.

7.4. LeWM Benchmarks

For experiments on **PushT**, **Two-Room**, **OG-Cube**, and **Reacher**, we build on top of the original implementation provided by [22] together with the provided expert rollouts for each task. We use a history size of three for all four tasks. As in [22], during episode sampling, we apply a 5-frame skip and group the consecutive actions between sampled frames into a single action vector.

8. Hyperparameter Search

For baselines trained on the game environments, we perform a grid search over full 10M episode runs. For SMWM [18], we grid-search the action-prediction loss weight; for LeWM [22], LeWM-Detached, LeWM-Time, and LeWM-Flat, we grid-search the SIGReg weight. The selected configuration is the final run with the lowest NMSE of an offline 5-step Residual MLP probe, averaged over all probing-table state groups. We use the probe of the predicted rollout $\hat{z}$ instead of the direct image embedding z , as we are interested in a world model that is able to predict future states accurately, not a pure image embedding. Results of the hyperparameter search and sensitivity are shown in Tab. 4 and Fig. 7.

9. Measuring Latent Path Straightness

For each environment, we estimate the path straightness shown in Tab. 2 by sampling $N = 100$ pairs of start and end states from the state variables defined in Sec. 7.2, with two exceptions: the score in Pong and the wood texture offset in Golf. We exclude the Pong score because its 0 to 99 range introduces non-continuous, looping visual features as the smaller digit rolls over. Similarly, the Golf wood texture offset is omitted because it produces texture crops that lack a meaningful linear structure. We then compute $T = 64$ linearly interpolated states for each trajectory. These states are rendered as images and encoded with the respective image encoder Enc_θ , yielding a tensor of latent trajectories $z \in \mathbb{R}^{N \times T \times 192}$. Finally, we compute the path straightness as the average ratio of the arc length to the chord length:

$$\frac{1}{N} \sum_{i=1}^N \frac{\sum_{t=1}^{T-1} \|z_{i,t+1} - z_{i,t}\|_2}{\|z_{i,T} - z_{i,1}\|_2} \quad (7)$$

We use this ratio because it is less sensitive to the choice of interpolation step size than cosine-based curvature, which depends on step granularity and is especially noisy for the small, low-resolution images in our setting.

10. State Probing on LeWorldModel Benchmarks with Static Wood-Texture Backgrounds

We evaluate the task-relevant physical quantities used for controlling each environment with offline MLP state probes. For **PushT**, we probe the 2D position of the agent, the 2D position of the T-shaped object, and the object orientation. For **TwoRoom**, we probe the 2D agent position. For **OG-Cube**, we probe the 3D positions and yaw angles of both the robot end-effector and the manipulated object. For **Reacher**, we probe the two robot joint angles, the 2D fingertip position, and the 2D target position.

Method	Search space	Selected		
		Pong	Dino	Golf
SMWM*	$\lambda_{\text{IDM}} \in \{0.1, 0.2, 0.5, 1, 2, 5, 10, 20, 50, 100\}$	100	0.2	1
LeWM*	$\lambda_{\text{SIGReg}} \in \{0.1, 0.2, 0.5, 1, 2, 5, 10, 20, 50, 100\}$	100	2	0.1
LeWM-Time*	$\lambda_{\text{SIGReg}} \in \{0.1, 0.2, 0.5, 1, 2, 5, 10, 20, 50, 100\}$	10	2	20
LeWM-Detached*	$\lambda_{\text{SIGReg}} \in \{0.1, 0.2, 0.5, 1, 2, 5, 10, 20, 50, 100\}$	0.2	20	0.02
LeWM-Flat*	$\lambda_{\text{SIGReg}} \in \{0.1, 0.2, 0.5, 1, 2, 5, 10, 20, 50, 100\}$	1	0.2	1
MotionJEPA (Ours)	$\lambda_z=0.25, \lambda_{\text{pred}}=0.5, \lambda_d=2$ (fixed)	all environments		

Table 4. **Hyperparameter Search Configurations and Selected Parameters.** We show the search space of each method together with the selected hyperparameters for each environment.

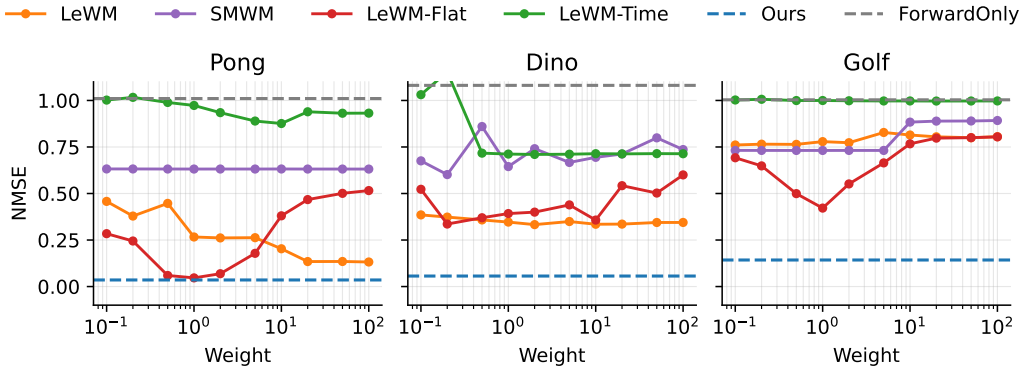


Figure 7. **Hyperparameter Search Visualization.** The NMSE loss is estimated from offline residual MLP probes on the predicted 5-step rollout $\hat{z}$.

For translational variables, we report the mean Euclidean distance in the native coordinates of the corresponding environment. For angular quantities, we report the mean absolute angular error in radians. Lower values are better in all cases. We compare LeWorldModel, LeWM-Flat, IDM, our method, and our method combined with IDM. For LeWorldModel, we use the original loss weights $\lambda_{\text{pred}} = 1$ and $\lambda_{\text{sigreg}} = 0.09$ [22]. LeWM-Flat applies SigReg after flattening the batch and temporal dimensions and uses $\lambda_{\text{sigreg}} = 1.0$, selected from the hyperparameter sweep on the game environments above. For IDM, we use $\lambda_{\text{pred}} = 1$ and $\lambda_{\text{IDM}} = 1$, as our hyperparameter search on the game environments above showed little sensitivity to the weighting. For our method, we use $\lambda_z = 0.25$, $\lambda_{\text{pred}} = 0.5$, and $\lambda_d = 2$.

Analysis. The probing results reveal substantial differences in the task-relevant physical information retained by the learned representations under static background distractors. LeWM performs poorly across several dynamic state variables, particularly on PushT, TwoRoom, and Reacher. LeWM-Flat improves recoverability in several cases, suggesting that stronger

regularization over flattened batch and temporal dimensions partially alleviates the problem, but a substantial gap remains to the dynamics-aware objectives. For example, on PushT the MLP agent-position error decreases from 121.17 for LeWM to 55.67 for LeWM-Flat, while our method reaches 8.60.

Our method substantially improves physical-state recoverability across the evaluated environments. The effect is especially pronounced on Reacher, where the linear probe obtains joint-angle errors of 0.0265 and 0.1246,rad and a fingertip-position error of 0.0241, compared with 1.468, 1.378,rad, and 0.169 for LeWM. The strong linear performance indicates that the controllable robot state is not only retained but also organized in a readily accessible form. IDM shows complementary behavior, providing particularly strong translational probing on PushT and OG-Cube, while remaining substantially weaker than our method on Reacher joint and fingertip states.

Combining our objective with IDM further improves several controllable variables, including TwoRoom agent position and Reacher joint and fingertip states. Overall, the results support the interpretation that static-background degradation is associated with the representation emphasizing highly predictable visual

(a) Offline Linear Probe NMSE $\downarrow$ of Image Embedding z

Method	Pong				Dino						Golf				
	avg.	ball	bats	score	avg.	cactus	clouds	dino	hearts	sky	avg.	bars	cross	ball	wood
ForwardOnly	0.965	0.999	1.000	0.899	0.911	1.002	0.999	1.001	1.001	0.239	1.002	1.002	1.001	1.002	1.004
Random	0.507	0.607	0.390	0.532	0.536	0.931	0.717	0.484	0.089	0.002	0.917	0.973	0.808	1.003	0.788
SMWM [18]	0.662	1.008	0.000	1.013	0.655	<u>0.643</u>	1.003	0.044	0.887	0.099	0.748	1.027	1.027	0.000	1.031
LeWM [22]	0.722	1.087	1.086	<u>0.025</u>	0.558	1.033	0.502	1.045	0.030	0.051	0.975	1.086	1.064	1.005	0.727
LeWM-Time	0.993	0.994	0.995	0.989	0.904	1.000	0.993	0.999	0.993	0.205	1.000	1.000	1.000	1.000	0.997
LeWM-Detached	0.411	0.706	0.429	0.126	0.762	1.082	0.858	1.054	0.149	0.204	0.985	1.020	0.778	1.035	0.979
LeWM-Flat	0.391	1.069	0.120	0.048	<u>0.445</u>	0.785	0.767	0.021	<u>0.026</u>	0.012	0.947	1.090	0.542	1.069	0.797
SMWM*	0.651	0.945	<u>0.001</u>	1.037	0.665	0.703	1.000	0.041	0.956	0.083	0.748	1.027	1.027	<u>0.000</u>	1.031
LeWM*	0.221	0.387	0.111	0.183	0.498	1.082	<u>0.362</u>	1.051	0.043	0.029	0.973	1.085	1.074	0.972	0.756
LeWM-Time*	1.014	1.002	1.015	1.024	0.969	0.790	<u>1.018</u>	0.900	1.003	1.013	1.011	1.004	0.986	1.022	1.020
LeWM-Detached*	0.251	0.538	0.105	0.137	0.539	1.083	0.422	1.049	0.089	0.096	0.982	1.090	1.079	1.003	0.749
LeWM-Flat*	<u>0.145</u>	<u>0.276</u>	0.077	0.096	0.454	0.927	0.742	<u>0.030</u>	0.041	0.031	<u>0.559</u>	<u>0.534</u>	0.202	0.496	0.846
MotionJEPA (Ours)	0.038	0.084	0.010	0.023	0.174	0.416	0.242	0.080	0.020	<u>0.011</u>	0.407	0.337	<u>0.422</u>	0.195	<u>0.746</u>

(b) Offline Linear Probe NMSE $\downarrow$ of Predicted Rollout $\hat{z}$

Method	Pong				Dino						Golf				
	avg.	ball	bats	score	avg.	cactus	clouds	dino	hearts	sky	avg.	bars	cross	ball	wood
ForwardOnly	1.000	1.002	1.001	0.997	0.999	1.001	1.002	1.000	1.003	0.979	1.002	1.002	1.001	1.000	1.004
Random	0.877	0.913	0.837	0.891	0.808	1.024	0.937	0.928	0.607	0.093	0.988	1.027	0.986	1.008	0.910
SMWM [18]	0.638	1.015	0.000	1.014	0.721	0.907	1.003	0.188	0.950	0.134	0.749	1.029	1.021	0.005	1.035
LeWM [22]	0.706	1.091	1.083	0.026	0.564	1.065	0.503	1.049	0.038	0.055	0.981	1.086	1.069	1.022	0.733
LeWM-Time	0.996	1.000	0.989	1.000	0.994	1.000	1.001	1.000	1.001	0.944	1.000	1.000	1.000	0.999	1.001
LeWM-Detached	0.418	0.778	0.436	0.129	0.764	1.065	0.861	1.046	0.170	0.215	1.005	1.048	0.894	1.025	0.973
LeWM-Flat	0.365	1.084	0.138	0.058	0.477	0.978	0.772	0.069	<u>0.030</u>	<u>0.018</u>	0.961	1.085	0.668	1.071	0.795
SMWM*	0.653	1.037	<u>0.001</u>	1.039	0.721	0.944	1.002	0.160	0.981	0.122	0.749	1.029	1.021	<u>0.005</u>	1.035
LeWM*	0.262	0.623	0.095	0.162	0.492	1.059	<u>0.352</u>	1.047	0.056	0.028	0.977	1.085	1.067	0.984	0.764
LeWM-Time*	1.003	0.959	1.022	1.017	0.947	<u>0.786</u>	1.009	0.832	0.984	1.003	1.011	1.011	1.010	1.010	1.013
LeWM-Detached*	0.274	0.682	0.110	0.137	0.526	1.057	0.400	1.043	0.104	0.085	0.984	1.084	1.066	1.020	0.751
LeWM-Flat*	<u>0.110</u>	<u>0.294</u>	0.027	0.058	<u>0.471</u>	0.987	0.742	<u>0.086</u>	0.044	0.038	<u>0.636</u>	<u>0.691</u>	0.387	0.492	0.838
MotionJEPA (Ours)	0.069	0.208	0.010	<u>0.026</u>	0.191	0.492	0.239	0.130	0.024	0.012	0.465	0.444	<u>0.525</u>	0.217	<u>0.745</u>

Table 5. **Offline linear probes.** Our method achieves the lowest average feature probing loss across all environments, indicating not only good feature coverage but also better linear separability.

content at the expense of task-relevant dynamics. Stronger SigReg alone, as in LeWM-Flat, helps but does not fully resolve this issue, while our objective and IDM provide stronger supervision for preserving dynamic physical state. Their combination further improves recoverability for several variables, consistent with the complementary planning gains observed in the main experiments.

10.1. Reconstruction-Based Visualization of Latent Information

To complement the quantitative probing results, we train a lightweight image decoder on top of each frozen latent representation as an offline visual probe. Figures 8 and 9 show representative reconstructions for all four environments. Within each row, original observations and their latent reconstructions are shown alternately, with methods ordered as LeWM, LeWM-Flat, IDM, ours, and

Ours+IDM.

The qualitative results are consistent with the state probing results. LeWM often reconstructs the dominant static appearance while only weakly recovering the task-relevant foreground state. LeWM-Flat improves foreground reconstruction in several environments, indicating that stronger regularization already alleviates part of this effect, but the recovered dynamic state remains less consistent than for the dynamics-aware objectives. IDM, Ours, and Ours+IDM generally preserve moving agents, manipulated objects, and articulated robot configurations more clearly. This difference is particularly visible on TwoRoom and OG-Cube, where LeWM reconstructions largely lose the foreground configuration, and on Reacher, where the dynamics-aware representations recover the robot pose substantially more clearly.

At the same time, our reconstructions still retain

Method	Probe	PushT			TwoRoom
		Agent pos. ↓	Object pos. ↓	Object angle ↓	Agent pos. ↓
LeWM	Linear	131.61	87.03	0.859	47.01
LeWM	MLP	121.17	73.66	0.788	45.33
LeWM-Flat	Linear	81.80	50.63	0.754	39.44
LeWM-Flat	MLP	55.67	28.88	0.419	37.01
IDM	Linear	3.99	24.47	0.670	3.71
IDM	MLP	3.66	18.01	0.413	3.29
Ours	Linear	17.46	27.33	0.763	6.34
Ours	MLP	8.60	18.20	0.397	1.48
Ours+IDM	Linear	3.58	26.52	0.726	1.55
Ours+IDM	MLP	4.32	17.12	0.367	0.73

Table 6. Physical-state probing on **PushT** and **TwoRoom** with a static wood-texture distractor. Translational quantities are reported as mean Euclidean error in the native environment coordinates, while angular error is the mean absolute angular error in radians. Lower is better, with the best MLP and Linear probe results in **bold**.

Method	Probe	OG-Cube				Reacher			
		Agent xyz	Object xyz	Agent yaw	Object yaw	Joint 0	Joint 1	Fingertip xy	Target xy
LeWM	Linear	0.182	0.187	1.394	1.436	1.468	1.378	0.169	0.124
LeWM	MLP	0.130	0.126	0.588	0.529	0.735	0.880	0.126	0.052
LeWM-Flat	Linear	0.132	0.151	1.417	1.508	0.292	1.078	0.126	0.125
LeWM-Flat	MLP	0.0162	0.0546	1.325	1.382	0.111	0.553	0.0738	0.121
IDM	Linear	0.0030	0.0068	1.371	1.539	1.385	0.895	0.168	0.125
IDM	MLP	0.0026	0.0046	1.274	1.522	1.162	0.820	0.166	0.125
Ours	Linear	0.0236	0.0214	1.403	1.473	0.0265	0.1246	0.0241	0.124
Ours	MLP	0.0113	0.0103	0.732	0.708	0.0141	0.0321	0.0061	0.0463
Ours+IDM	Linear	0.0040	0.0089	1.374	1.482	0.0139	0.0392	0.0107	0.124
Ours+IDM	MLP	0.0048	0.0059	0.808	0.841	0.0100	0.0234	0.0042	0.0487

Table 7. Physical-state probing on **OG-Cube** and **Reacher** with a static wood-texture distractor. Position errors are mean Euclidean errors, while angular quantities are mean absolute errors in radians. Lower is better, with the best MLP and Linear probe results in **bold**.

coarse scene appearance rather than becoming invariant to static visual content. Fine wood-texture details are largely smoothed out, while broad background structure remains recoverable. This is desirable because static or slowly varying scene information may still encode useful context such as goals or obstacles. Together with the quantitative probes, these results suggest that the proposed objective reduces the dominance of highly predictable static appearance while preserving both task-relevant dynamics and coarse contextual information.

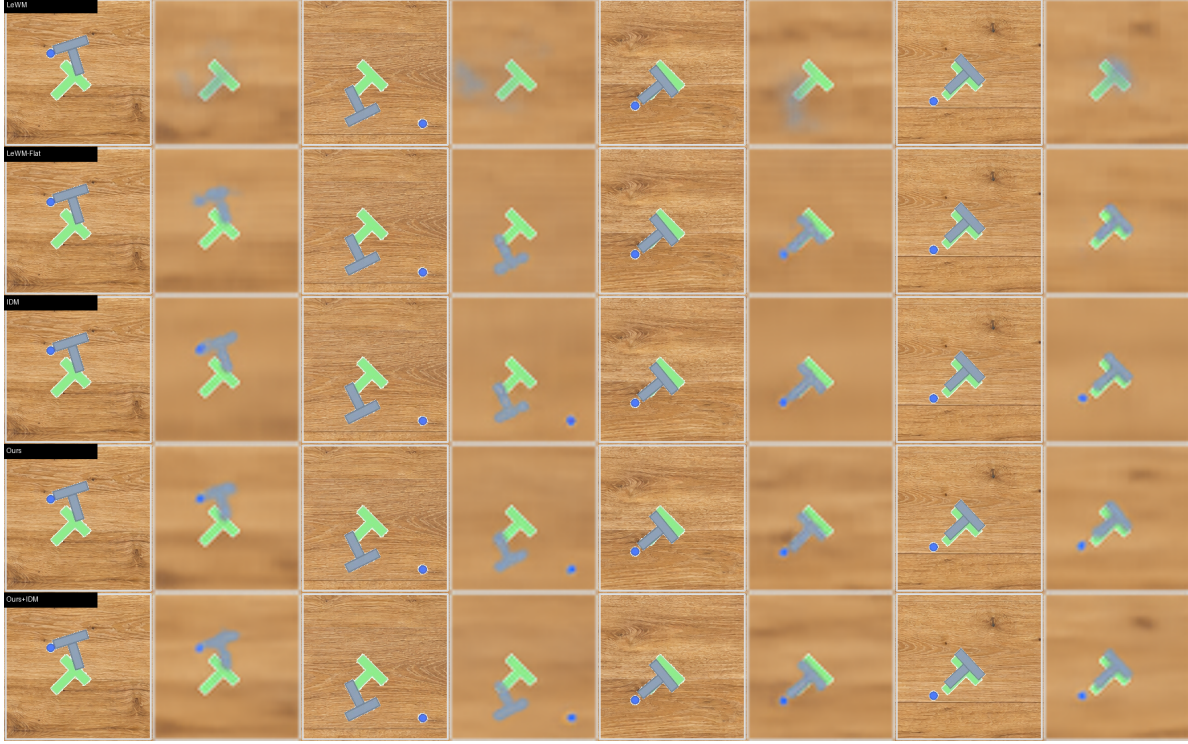
(a) Offline Residual MLP Probe NMSE $\downarrow$ of Image Embedding z

Method	avg.	Pong		
		ball	bats	score
ForwardOnly	1.058 ± 0.099	1.260 ± 0.449	1.001 ± 0.001	0.930 ± 0.121
Random	0.116 ± 0.009	0.143 ± 0.014	0.093 ± 0.011	0.115 ± 0.005
SMWM [18]	0.657 ± 0.004	1.001 ± 0.001	0.002 ± 0.000	1.002 ± 0.001
LeWM [22]	0.658 ± 0.003	0.999 ± 0.004	1.002 ± 0.000	0.003 ± 0.001
LeWM-Time	1.077 ± 0.096	1.100 ± 0.145	1.071 ± 0.096	1.063 ± 0.066
LeWM-Detached	0.171 ± 0.092	0.174 ± 0.013	0.331 ± 0.255	0.007 ± 0.000
LeWM-Flat	0.284 ± 0.006	0.890 ± 0.003	0.014 ± 0.011	0.005 ± 0.000
SMWM*	0.651 ± 0.007	0.983 ± 0.029	0.002 ± 0.001	1.003 ± 0.001
LeWM*	0.061 ± 0.008	0.163 ± 0.025	0.018 ± 0.002	0.012 ± 0.000
LeWM-Time*	0.687 ± 0.006	0.022 ± 0.004	0.972 ± 0.017	1.002 ± 0.001
LeWM-Detached*	0.048 ± 0.003	0.103 ± 0.008	0.038 ± 0.002	0.008 ± 0.001
LeWM-Flat*	0.008 ± 0.001	0.012 ± 0.001	0.004 ± 0.001	0.008 ± 0.001
MotionJEPA (Ours)	0.004 ± 0.000	0.005 ± 0.001	0.003 ± 0.001	0.004 ± 0.001

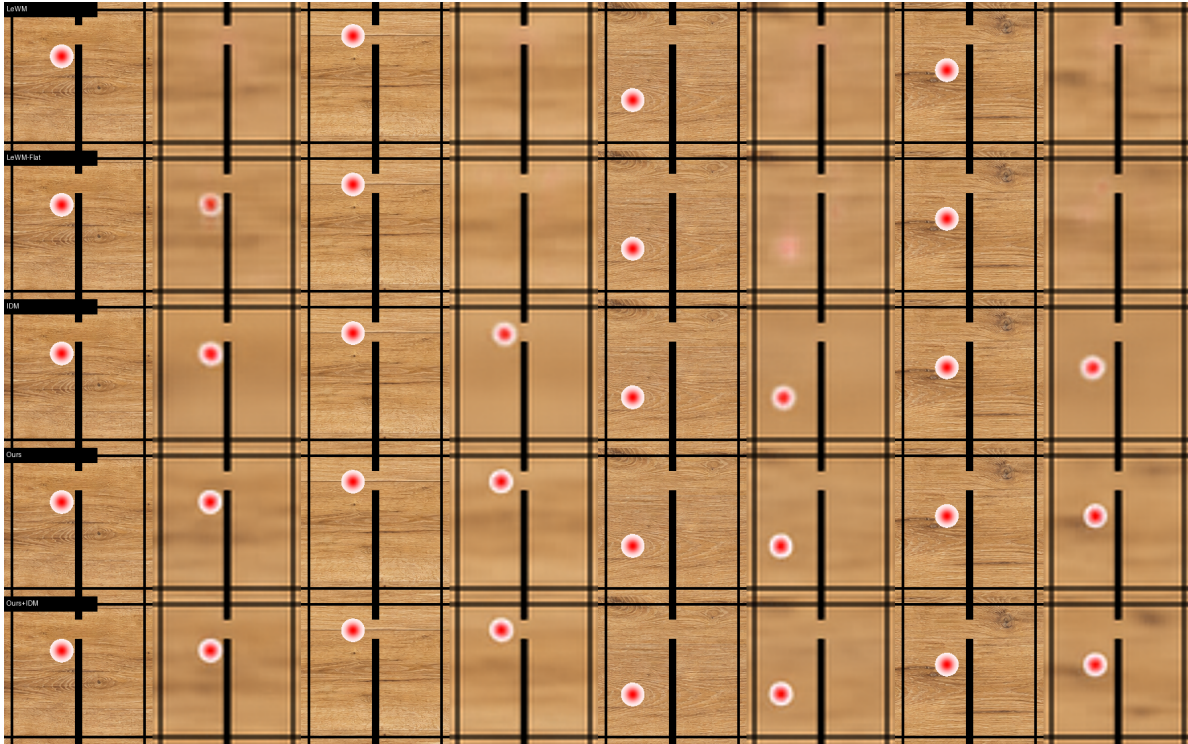
(b) Offline Residual MLP Probe NMSE $\downarrow$ of Predicted Rollout $\hat{z}$

Method	avg.	Pong		
		ball	bats	score
ForwardOnly	1.010 ± 0.015	1.041 ± 0.068	1.006 ± 0.008	0.992 ± 0.017
Random	0.554 ± 0.022	0.721 ± 0.031	0.478 ± 0.028	0.507 ± 0.019
SMWM [18]	0.630 ± 0.003	1.001 ± 0.001	0.002 ± 0.000	1.002 ± 0.001
LeWM [22]	0.644 ± 0.003	1.002 ± 0.000	1.002 ± 0.001	0.002 ± 0.000
LeWM-Time	1.001 ± 0.008	1.003 ± 0.002	0.998 ± 0.019	1.002 ± 0.002
LeWM-Detached	0.239 ± 0.095	0.397 ± 0.005	0.347 ± 0.258	0.007 ± 0.001
LeWM-Flat	0.278 ± 0.013	0.968 ± 0.017	0.039 ± 0.035	0.005 ± 0.000
SMWM*	0.630 ± 0.002	1.001 ± 0.001	0.001 ± 0.000	1.002 ± 0.001
LeWM*	0.138 ± 0.005	0.471 ± 0.019	0.020 ± 0.000	0.009 ± 0.001
LeWM-Time*	0.868 ± 0.028	0.507 ± 0.101	1.002 ± 0.001	1.002 ± 0.001
LeWM-Detached*	0.129 ± 0.011	0.395 ± 0.037	0.053 ± 0.004	0.008 ± 0.001
LeWM-Flat*	0.054 ± 0.003	0.183 ± 0.016	0.004 ± 0.001	0.007 ± 0.002
MotionJEPA (Ours)	0.036 ± 0.001	0.122 ± 0.003	0.004 ± 0.000	0.005 ± 0.001

Table 8. Pong Standard Deviation of Three Seed Runs.



(a) PushT



(b) TwoRoom

Figure 8. **Qualitative reconstruction from frozen latent representations on PushT and TwoRoom.** Rows within each environment correspond to LeWM, LeWM-Flat, IDM, our method, and Ours+IDM, with original observations and their reconstructions shown alternately. LeWM often reconstructs the dominant static appearance while only weakly recovering the task-relevant foreground. LeWM-Flat improves foreground recovery, while IDM, our method, and Ours+IDM generally preserve the moving entities more clearly.



(a) OG-Cube



(b) Reacher

Figure 9. **Qualitative reconstruction from frozen latent representations on OG-Cube and Reacher.** Rows within each environment correspond to LeWM, LeWM-Flat, IDM, our method, and Ours+IDM, with original observations and their reconstructions shown alternately. Dynamics-aware representations recover the articulated foreground state substantially more clearly than LeWM, while LeWM-Flat provides an intermediate improvement. This is particularly visible on Reacher, where the robot configuration is more faithfully reconstructed despite the dominant wood-texture background.

(a) Offline Residual MLP Probe NMSE $\downarrow$ of Image Embedding z

Method	avg.	Dino				
		cactus	clouds	dino	hearts	sky
ForwardOnly	0.949 ± 0.054	1.003 ± 0.004	1.002 ± 0.010	1.002 ± 0.003	1.031 ± 0.045	0.511 ± 0.422
Random	0.361 ± 0.006	0.833 ± 0.010	0.535 ± 0.016	0.116 ± 0.005	0.031 ± 0.002	0.004 ± 0.002
SMWM [18]	0.649 ± 0.040	0.434 ± 0.213	1.075 ± 0.061	0.028 ± 0.028	0.787 ± 0.061	0.071 ± 0.057
LeWM [22]	0.394 ± 0.032	0.769 ± 0.304	0.248 ± 0.073	0.983 ± 0.033	0.012 ± 0.002	0.005 ± 0.002
LeWM-Time	0.913 ± 0.019	1.003 ± 0.001	0.992 ± 0.013	1.002 ± 0.002	1.029 ± 0.034	0.248 ± 0.130
LeWM-Detached	0.645 ± 0.020	1.002 ± 0.002	0.727 ± 0.043	1.002 ± 0.002	0.017 ± 0.007	0.006 ± 0.002
LeWM-Flat	0.331 ± 0.008	0.522 ± 0.073	0.593 ± 0.026	0.002 ± 0.000	0.006 ± 0.001	0.007 ± 0.001
SMWM*	0.606 ± 0.042	0.398 ± 0.047	1.005 ± 0.038	0.003 ± 0.000	0.775 ± 0.177	0.069 ± 0.080
LeWM*	0.282 ± 0.013	0.990 ± 0.008	0.059 ± 0.006	0.751 ± 0.072	0.011 ± 0.002	0.012 ± 0.002
LeWM-Time*	0.697 ± 0.002	0.005 ± 0.001	1.004 ± 0.002	0.003 ± 0.001	0.971 ± 0.007	1.004 ± 0.002
LeWM-Detached*	0.332 ± 0.004	1.005 ± 0.002	0.073 ± 0.003	0.959 ± 0.011	0.011 ± 0.004	0.017 ± 0.002
LeWM-Flat*	0.301 ± 0.008	0.650 ± 0.128	0.498 ± 0.033	0.002 ± 0.000	0.007 ± 0.001	0.016 ± 0.003
MotionJEPa (Ours)	0.021 ± 0.002	0.011 ± 0.002	0.037 ± 0.003	0.006 ± 0.003	0.006 ± 0.002	0.004 ± 0.001

(b) Offline Residual MLP Probe NMSE $\downarrow$ of Predicted Rollout $\hat{z}$

Method	avg.	Dino				
		cactus	clouds	dino	hearts	sky
ForwardOnly	1.081 ± 0.067	1.019 ± 0.016	1.006 ± 0.005	1.004 ± 0.004	1.017 ± 0.024	1.634 ± 0.537
Random	0.617 ± 0.032	0.974 ± 0.008	0.764 ± 0.008	0.631 ± 0.042	0.279 ± 0.212	0.026 ± 0.012
SMWM [18]	0.686 ± 0.046	0.839 ± 0.139	1.002 ± 0.003	0.092 ± 0.033	0.933 ± 0.186	0.076 ± 0.064
LeWM [22]	0.407 ± 0.024	0.856 ± 0.143	0.251 ± 0.065	0.995 ± 0.010	0.017 ± 0.003	0.005 ± 0.002
LeWM-Time	1.106 ± 0.103	1.011 ± 0.009	1.009 ± 0.005	1.003 ± 0.002	1.026 ± 0.036	1.845 ± 0.860
LeWM-Detached	0.646 ± 0.016	0.988 ± 0.002	0.732 ± 0.035	1.002 ± 0.001	0.028 ± 0.007	0.006 ± 0.002
LeWM-Flat	0.396 ± 0.009	0.877 ± 0.055	0.633 ± 0.028	0.055 ± 0.006	0.009 ± 0.001	0.007 ± 0.001
SMWM*	0.671 ± 0.015	0.801 ± 0.022	0.999 ± 0.005	0.075 ± 0.013	0.878 ± 0.081	0.079 ± 0.072
LeWM*	0.334 ± 0.003	0.991 ± 0.002	0.060 ± 0.006	1.003 ± 0.001	0.018 ± 0.001	0.012 ± 0.002
LeWM-Time*	0.710 ± 0.001	0.120 ± 0.002	1.004 ± 0.002	0.020 ± 0.000	0.959 ± 0.009	1.003 ± 0.002
LeWM-Detached*	0.343 ± 0.004	0.988 ± 0.005	0.078 ± 0.008	1.002 ± 0.002	0.020 ± 0.003	0.014 ± 0.003
LeWM-Flat*	0.353 ± 0.021	0.923 ± 0.027	0.524 ± 0.056	0.066 ± 0.007	0.013 ± 0.003	0.018 ± 0.004
MotionJEPa (Ours)	0.055 ± 0.001	0.243 ± 0.012	0.040 ± 0.001	0.051 ± 0.004	0.007 ± 0.001	0.005 ± 0.001

Table 9. Dino Standard Deviation of Three Seed Runs.

(a) Offline Residual MLP Probe NMSE $\downarrow$ of Image Embedding z					
Method	avg.	bars	Golf cross	ball	wood
ForwardOnly	1.980 ± 0.769	1.233 ± 0.200	1.558 ± 0.866	1.077 ± 0.108	4.324 ± 2.622
Random	0.776 ± 0.006	0.857 ± 0.011	0.296 ± 0.018	0.989 ± 0.002	0.653 ± 0.002
SMWM [18]	0.727 ± 0.002	1.000 ± 0.001	0.986 ± 0.001	0.002 ± 0.001	1.003 ± 0.001
LeWM [22]	0.763 ± 0.009	1.002 ± 0.001	1.001 ± 0.000	0.942 ± 0.061	0.084 ± 0.033
LeWM-Time	1.053 ± 0.076	1.008 ± 0.005	1.014 ± 0.025	1.004 ± 0.002	1.195 ± 0.302
LeWM-Detached	0.811 ± 0.100	0.924 ± 0.074	0.683 ± 0.532	0.930 ± 0.037	0.572 ± 0.126
LeWM-Flat	0.694 ± 0.066	1.002 ± 0.001	0.379 ± 0.539	0.993 ± 0.013	0.052 ± 0.025
SMWM*	0.727 ± 0.002	1.000 ± 0.001	0.986 ± 0.001	0.002 ± 0.001	1.003 ± 0.001
LeWM*	0.760 ± 0.007	1.001 ± 0.001	1.002 ± 0.001	0.909 ± 0.084	0.110 ± 0.063
LeWM-Time*	0.985 ± 0.005	0.972 ± 0.013	0.992 ± 0.015	0.984 ± 0.002	1.002 ± 0.001
LeWM-Detached*	0.745 ± 0.008	1.002 ± 0.001	1.002 ± 0.001	0.908 ± 0.022	0.048 ± 0.015
LeWM-Flat*	0.299 ± 0.027	0.317 ± 0.058	0.059 ± 0.018	0.312 ± 0.066	0.377 ± 0.044
MotionJEPa (Ours)	0.061 ± 0.012	0.095 ± 0.012	0.083 ± 0.004	0.033 ± 0.023	0.033 ± 0.008

(b) Offline Residual MLP Probe NMSE $\downarrow$ of Predicted Rollout $\hat{z}$					
Method	avg.	bars	Golf cross	ball	wood
ForwardOnly	1.003 ± 0.003	1.002 ± 0.001	1.001 ± 0.001	1.005 ± 0.008	1.005 ± 0.004
Random	0.934 ± 0.004	0.995 ± 0.001	0.913 ± 0.007	0.998 ± 0.002	0.781 ± 0.014
SMWM [18]	0.729 ± 0.002	1.001 ± 0.000	1.001 ± 0.000	0.006 ± 0.001	1.002 ± 0.001
LeWM [22]	0.767 ± 0.011	1.002 ± 0.000	1.001 ± 0.001	0.947 ± 0.063	0.094 ± 0.034
LeWM-Time	1.003 ± 0.004	1.006 ± 0.005	1.006 ± 0.008	1.000 ± 0.002	1.002 ± 0.002
LeWM-Detached	0.855 ± 0.054	0.967 ± 0.038	0.821 ± 0.313	0.930 ± 0.045	0.618 ± 0.102
LeWM-Flat	0.722 ± 0.045	1.002 ± 0.001	0.590 ± 0.356	0.994 ± 0.014	0.061 ± 0.030
SMWM*	0.729 ± 0.002	1.001 ± 0.000	1.001 ± 0.000	0.006 ± 0.001	1.002 ± 0.001
LeWM*	0.763 ± 0.007	1.002 ± 0.001	1.002 ± 0.001	0.913 ± 0.080	0.117 ± 0.063
LeWM-Time*	0.997 ± 0.000	0.997 ± 0.002	0.995 ± 0.010	0.991 ± 0.001	1.002 ± 0.002
LeWM-Detached*	0.746 ± 0.008	1.002 ± 0.001	1.001 ± 0.001	0.909 ± 0.020	0.051 ± 0.017
LeWM-Flat*	0.448 ± 0.030	0.540 ± 0.048	0.320 ± 0.044	0.402 ± 0.062	0.425 ± 0.048
MotionJEPa (Ours)	0.164 ± 0.025	0.262 ± 0.029	0.348 ± 0.005	0.065 ± 0.049	0.037 ± 0.006

Table 10. Golf Standard Deviation of Three Seed Runs.

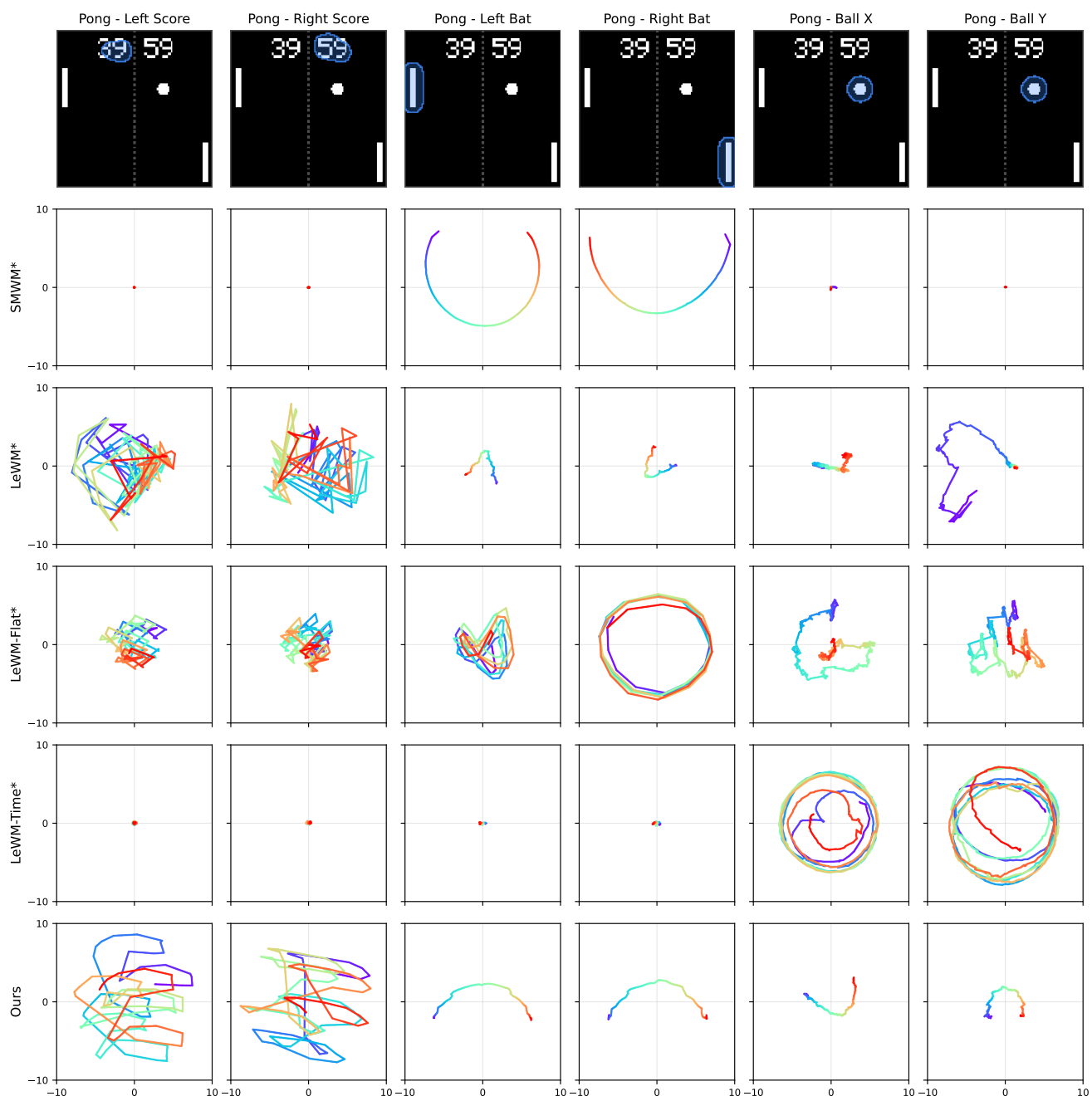


Figure 10. Per-Feature Latent Trajectories Visualization for Pong.

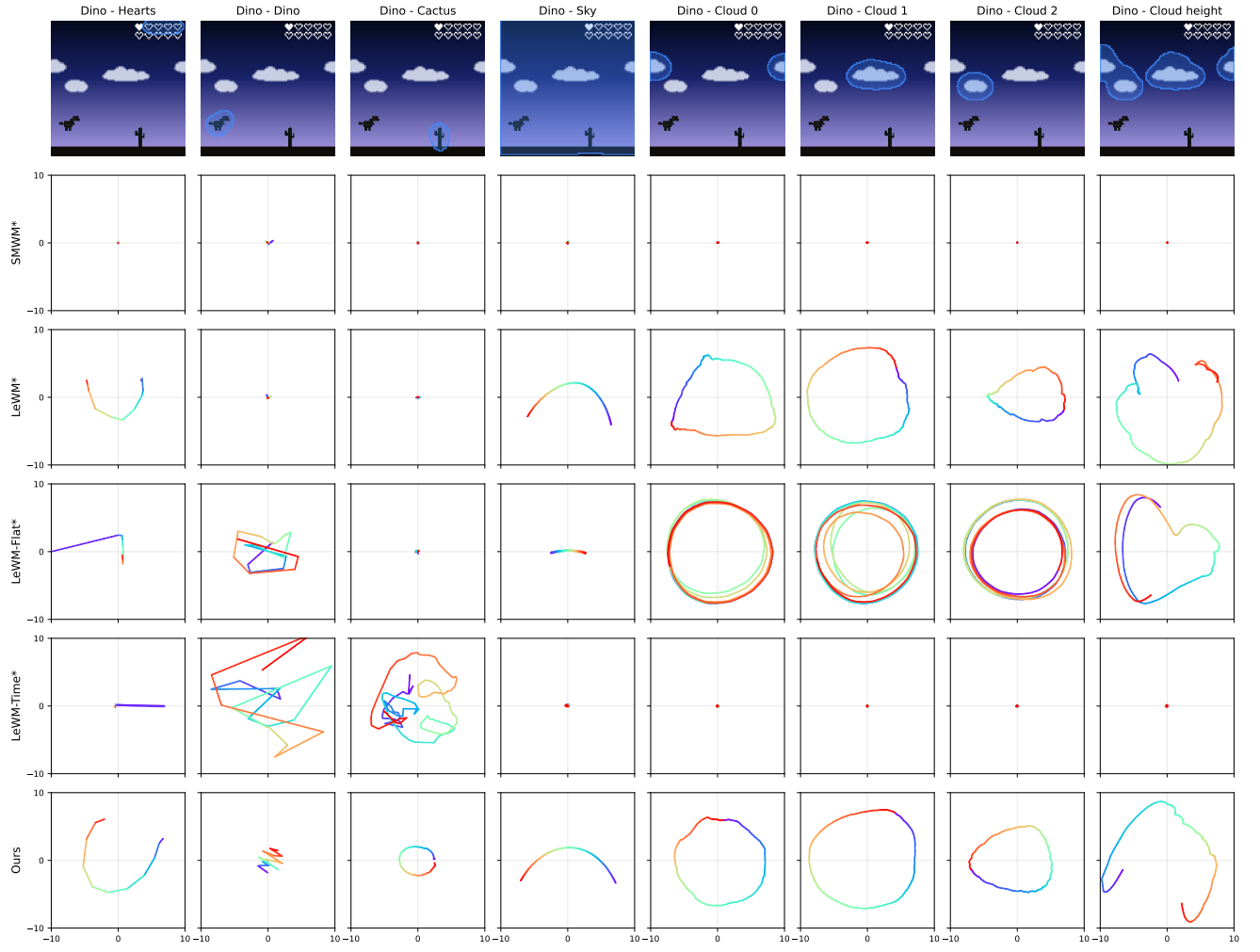


Figure 11. Per-Feature Latent Trajectories Visualization for Dino.

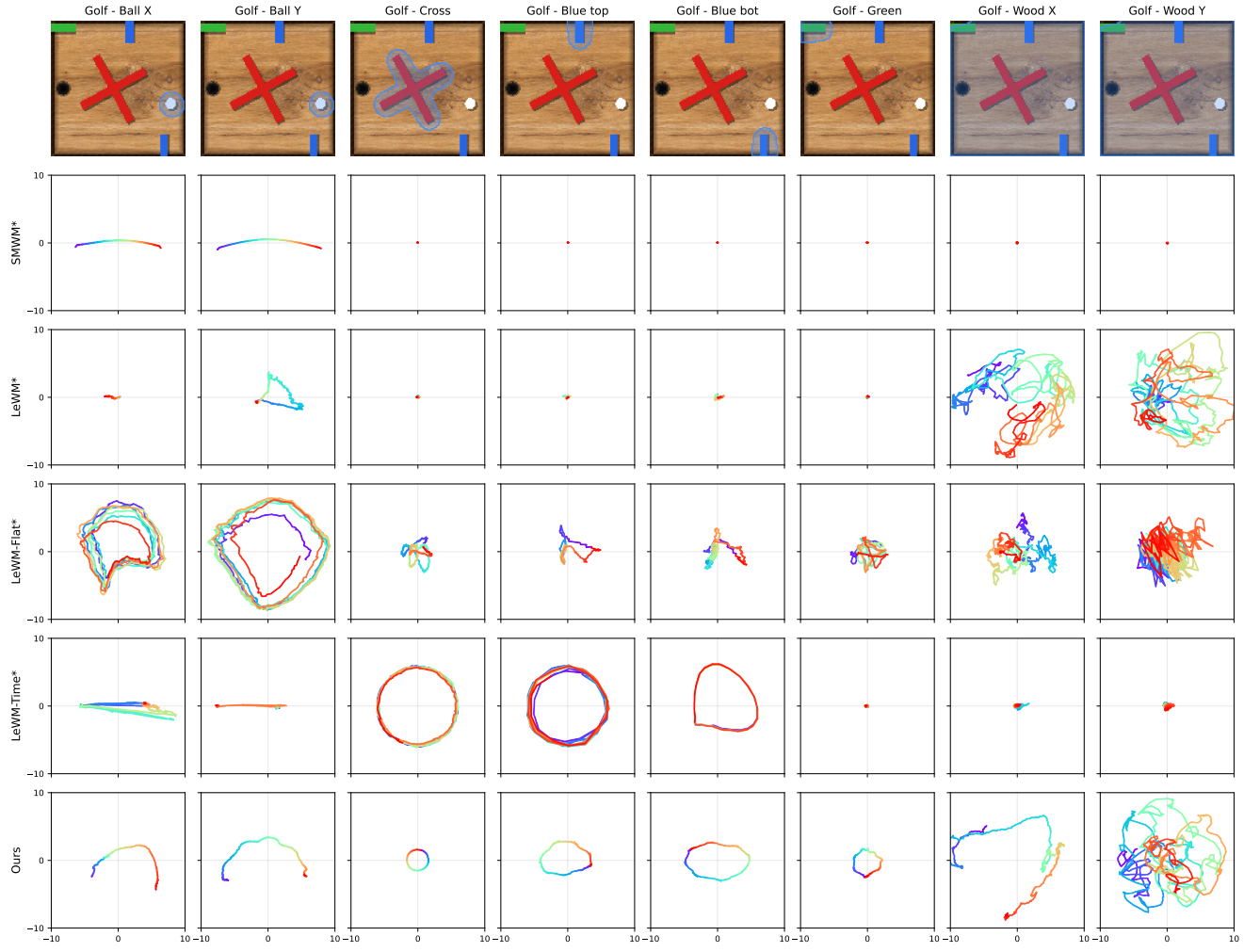


Figure 12. Per-Feature Latent Trajectories Visualization for Golf.